DAEN (ISEN) 489 Final Project

Due date: May 1, 2026 (online)

Note: Students may work in teams of up to three members. Each team should collaborate internally on all aspects of the project, including modeling, implementation, and analysis. Collaboration between different teams is not permitted. In particular, teams must not share code, data processing pipelines, or detailed solution approaches. Each team is expected to submit original work. High-level discussions of general concepts are allowed; however, any exchange of implementation details or specific solution strategies across teams is prohibited. All submitted materials must reflect the independent work of the team.

1 Overview

In this project, you will design and evaluate a sequential decision-making strategy for managing a fantasy English Premier League (EPL) team. The key challenge is that the environment is *not directly available*: player performance must be *learned from data*. This leads naturally to a two-layer structure:

1. **Prediction:** learn a model of player rewards,
2. **Control:** optimize decisions using this model.

This project illustrates a central theme in modern reinforcement learning: control with learned models of the environment.

2 MDP Formulation

We consider a finite-horizon MDP with horizon T (number of game-weeks). The state is $s_t = (\mathcal{S}_t, B_t, \phi_t, t)$ where:

- $\mathcal{S}_t$ is the current squad,
- B_t is remaining budget,
- ϕ_t denotes observable features (player statistics),
- t is the gameweek.

The action space is $a_t \in \mathcal{A}(s_t)$, where actions include:

- $a_t = 0$ (no transfer),
- $a_t = (i \to j)$ replacing player i with j.

The true reward is $R_t = \sum_{i \in \mathcal{S}_t} r_{i,t}$. Since $r_{i,t}$ is unknown at decision time, we estimate: $\hat{r}_{i,t} = f_\theta(x_{i,t})$. where $x_{i,t}$ is a feature vector

$$x_{i,t} = \phi(i, t, \mathcal{H}_{t-1}),$$

where $\mathcal{H}_{t-1}$ denotes all information available up to week $t-1$. The function ϕ extracts observable features such as recent performance, price, and contextual variables (e.g., opponent strength) specific

to player i. For example,

$$x_{i,t} = \begin{bmatrix} \text{price}_{i,t} \\ \text{form}_{i,t} \\ \text{minutes_form}_{i,t} \\ \text{goals}_{i,t-1:t-k} \\ \text{assists}_{i,t-1:t-k} \\ \text{home}_{i,t} \\ \text{opp_strength}_{i,t} \\ \text{position}_i \end{bmatrix}$$

The dynamics are given by $s_{t+1} \sim P(\cdot|s_t, a_t)$ where transitions depend on squad updates, exogenous player evolution etc. In this project, your NOT expected to explicitly estimate the transition kernel. Instead, the system is modeled using a squad and budget updates which are deterministic given the action and player features evolve according to observed data $\mathcal{D}$.
The objective is:

$$\max_{\pi} \mathbb{E}\Big[\sum_{t=1}^{T} R_t\Big].$$

Define the value function:

$$V_t(s) = \max_{\pi} \mathbb{E}\Big[\sum_{k=t}^{T} R_k \mid s_t = s\Big]$$

The Bellman recursion is:

$$V_t(s) = \max_{a \in \mathcal{A}(s)} \left\{R_t(s,a) + \mathbb{E}_{s' \sim P(\cdot|s,a)}[V_{t+1}(s')]\right\}.$$

Define the Q-function:

$$Q_t(s,a) = R_t(s,a) + \mathbb{E}_{s' \sim P(\cdot|s,a)}[V_{t+1}(s')].$$

3 Tasks

The project consists of two tasks:

1. Construct features from historical data and train a reward model.
2. Implement RL algorithms based upon observed data and reward model.
3. Fine-tune policy model.

You will submit a python code for your *policy* say π. The instructor will evaluate it in a hidden environment with N rollouts:

$$V(\pi) = \frac{1}{N} \sum_{k=1}^{N} \sum_{t=1}^{T} R_t^{(k)}.$$

4 Numerical Constraints

Each team consists of 15 players with fixed positional composition: 2 goalkeepers, 5 defenders, 5 midfielders, and 3 forwards. Managers operate under a total budget of £100 million, with individual player prices typically ranging from £4.0 to about £14.5. Each gameweek, managers may make one free transfer, while additional transfers incur a penalty of -4 points each (**no carry over of free transfer**). A maximum of three players may be selected from any single real-world club. In addition, a starting lineup of 11 players must be selected subject to a formation constraint (at least 3 defenders, 2 midfielders, and 1 forward), and one player is designated as captain, earning double points for that week.

5 Policy Format

Students must submit a file named `policy.py` (see example below) containing a class `StudentPolicy` with the methods `fit(train_data)`, `reset()`, and `act(state)`.

- `fit(train_data)` should train the student's reward model using the historical dataset provided for the project.
- `reset()` is called once at the start of each rollout.
- `act(state)` must return a list of transfers of the form $[(i_1, j_1), \ldots, (i_m, j_m)]$, where each pair (i_ℓ, j_ℓ) denotes removing player i_ℓ from the current squad and adding player j_ℓ. Returning `[]` denotes no transfer.

```python
# policy.py
import numpy as np
import pandas as pd

class StudentPolicy:
    def __init__(self):
        self.model = None
        self.feature_cols = None

    def fit(self, train_data):
        """
        Fit a reward prediction model using historical data.
        Students should replace this baseline with their own model.
        """
        # Example placeholder
        self.feature_cols = ["price", "form", "minutes_form"]
        X = train_data[self.feature_cols].fillna(0.0).values
        y = train_data["points"].values

        # Dummy baseline: predict the mean
        self.model = {"mean_points": float(np.mean(y))}

    def reset(self):
        """
        Reset any rollout-specific memory.
        """
        pass

    def predict_reward(self, player_dict):
        """
        Predict reward for one player from current features.
        """
        if self.model is None:
            raise ValueError("Model has not been fit yet.")

        # Dummy baseline
        return self.model["mean_points"]

    def act(self, state):
        """
        Return a list of transfers [(player_out_id, player_in_id), ...]
        """
        return []
```

6 Deliverables and Grading

Python code for policy, Report (4–6 pages), Policy implementation The grading will be as follows:

- Modeling (30%)
- Implementation (25%)
- Methodology (20%)
- Experiments (15%)
- Policy performance (10%)

A Data

```
url =
↪  "https://raw.githubusercontent.com/vaastav/Fantasy-Premier-League/master/data/2023-24/gws/merged_gw.csv"
df = pd.read_csv(url)
```

B Python Template

```
import numpy as np
import pandas as pd

class PlayerSimulator:
    def __init__(self, model, noise_std=1.0):
        self.model = model
        self.noise_std = noise_std

    def predict_points(self, features):
        pred = self.model.predict(features.reshape(1, -1))[0]
        noise = np.random.normal(0, self.noise_std)
        return max(0, pred + noise)


class FPLEnv:
    def __init__(self, data, model, squad_size=5):
        self.data = data
        self.model = model
        self.sim = PlayerSimulator(model)
        self.players = data["player_id"].unique()
        self.squad_size = squad_size

    def reset(self):
        self.week = self.data["gameweek"].min()
        self.squad = np.random.choice(self.players, self.squad_size, replace=False)
        return self._get_state()

    def _get_state(self):
        return (self.week, tuple(self.squad))

    def _get_features(self, player_id):
        row = self.data[
            (self.data["player_id"] == player_id) &
            (self.data["gameweek"] == self.week)
        ]
        if len(row) == 0:
            return None
        return row[["price","form","minutes_form"]].values[0]
```

```python
    def step(self, action):
        if action is not None:
            p_out, p_in = action
            self.squad = np.array([
                p if p != p_out else p_in for p in self.squad
            ])

        reward = 0
        for p in self.squad:
            feat = self._get_features(p)
            if feat is not None:
                reward += self.sim.predict_points(feat)

        self.week += 1
        done = self.week > self.data["gameweek"].max()

        return self._get_state(), reward, done


def run_no_transfer(env):
    state = env.reset()
    total_reward = 0

    done = False
    while not done:
        state, reward, done = env.step(None)
        total_reward += reward

    return total_reward
```